Image Deblurring via Total Variation Regularization

A Convex Optimisation Approach

Reynaldi Aditama, Nam Bui, David Ou

March – April 2026

Contents

1 Introduction **2**

2 Mathematical Framework **2**
2.1 Total Variation of an Image 2
2.2 The TV/L^2 Variational Model 3
2.3 Why Total Variation, Not Tikhonov? 3

3 The Blurring Operator **4**
3.1 Gaussian Convolution 4
3.2 Building the Blur Matrix H 4
3.3 Checking the Degradation 5

4 The Optimisation Problem and Its Convexity **5**
4.1 Convexity 5
4.2 Existence of a Solution 5
4.3 Role of the Regularisation Parameter λ 5

5 Implementation **6**
5.1 Experimental Setup 6
5.2 CVXPY Formulation 6
5.3 Computational Notes 6

6 Results and Analysis **7**
6.1 Solver Output 7
6.2 Visual Comparison Across λ Values 7

7 Discussion **9**
7.1 Limitations of the Model 9
7.2 Extensions 9

8 Conclusion **10**

1 Introduction

Optical blurring is unavoidable in real imaging systems. Camera shake, diffraction limits, atmospheric turbulence, and out-of-focus optics all introduce a spatially averaged degradation that obscures fine detail. The goal of image deblurring—also called *image deconvolution*—is to reverse this process and recover a sharp estimate of the original scene.

Mathematically, the formation model is

$$\mathbf{f} = H\mathbf{u}_0 + \boldsymbol{\eta}, \tag{1}$$

where $\mathbf{u}_0 \in \mathbb{R}^n$ is the vectorised sharp image ($n = N^2$ for an $N \times N$ image), $H \in \mathbb{R}^{n\times n}$ is the blurring operator, and $\boldsymbol{\eta} \in \mathbb{R}^n$ is additive noise. Given $\mathbf{f}$ and H, the task is to estimate $\mathbf{u}_0$.

The straightforward approach—minimising the residual $\|H\mathbf{u} - \mathbf{f}\|_2^2$ over $\mathbf{u}$—is known as the *inverse filter*. Because typical blurring kernels suppress high spatial frequencies, the matrix H is severely ill-conditioned, and any small noise component in $\mathbf{f}$ is amplified catastrophically in the inversion. This instability forces us to incorporate *a priori* knowledge about what a natural image should look like, a technique broadly called *regularisation*.

Tikhonov regularisation [3] penalises the squared ℓ^2 norm of the gradient, which stabilises the inversion but tends to produce overly smooth reconstructions with blurred edges. A more image-faithful alternative, introduced by Rudin, Osher, and Fatemi [1], is *total variation* (TV) regularisation. TV measures the ℓ^1 norm of the gradient and acts as an edge-preserving smoother: it removes noise and deconvolves the blur while leaving step-like discontinuities intact.

This report studies the discrete TV/L^2 model for image deblurring. Section 2 develops the mathematical framework from first principles, Section 3 explains how the blurring operator—including the explicit sparse-matrix construction—is derived, Section 4 discusses the optimisation problem and its convexity, Section 5 describes the numerical implementation, and Section 6 presents and analyses the experimental results.

2 Mathematical Framework

2.1 Total Variation of an Image

For a continuous function $u : \Omega \subset \mathbb{R}^2 \to \mathbb{R}$, the total variation is

$$\mathrm{TV}(u) = \int_\Omega \|\nabla u(x, y)\|_2 \, dx\, dy.$$

The key feature of this functional is that it counts the total amount of oscillation in u without penalising the individual jump heights. A piecewise-constant function with finitely many jumps has finite total variation, which is why the TV functional was identified early on as the appropriate norm for images [1].

In a discrete setting, the image $u : \{1, \ldots, N\}^2 \to [0, 1]$ is an $N \times N$ matrix of pixel intensities. The discrete gradient at pixel (i, j) is the two-component vector of forward differences

$$\nabla u_{i,j} = \begin{pmatrix} u_{i+1,j} - u_{i,j} \\ u_{i,j+1} - u_{i,j} \end{pmatrix} \in \mathbb{R}^2, \tag{2}$$

with the convention that differences wrap around at the boundary (periodic boundary conditions). The *isotropic* discrete total variation is then

$$\mathrm{TV}(u) = \sum_{i=1}^{N}\sum_{j=1}^{N} \|\nabla u_{i,j}\|_2 = \sum_{i,j} \sqrt{(u_{i+1,j} - u_{i,j})^2 + (u_{i,j+1} - u_{i,j})^2}. \tag{3}$$

The choice of the ℓ^2 norm within each pixel gradient—as opposed to the ℓ^1 norm used in *anisotropic* TV—makes the functional rotationally invariant, meaning it treats horizontal and diagonal edges on an equal footing. This is the form used throughout this project.

Remark 2.1. *The TV functional is convex but* not *strictly convex and* not *differentiable at points where $\nabla u_{i,j} = \mathbf{0}$. The subdifferential at the origin is the closed unit ball, so the functional is subdifferentiable everywhere. This non-smoothness is precisely what allows TV to promote piecewise-constant solutions: the subdifferential at a flat region contains the zero vector, so a smooth optimiser is not forced to move away from it.*

2.2 The TV/L^2 Variational Model

Let $\mathbf{u} \in \mathbb{R}^n$ denote the vectorised form of the $N \times N$ image (pixels stacked row-by-row), and let $U \in \mathbb{R}^{N\times N}$ denote the corresponding matrix form. The TV/L^2 deblurring model is

$$\min_{\mathbf{u}} \ \underbrace{\|H\mathbf{u} - \mathbf{f}\|_2^2}_{\text{data fidelity}} + \lambda \underbrace{\mathrm{TV}(U)}_{\text{regulariser}} \quad \text{subject to} \quad 0 \le \mathbf{u} \le 1. \tag{4}$$

The two terms encode competing objectives:

- The **data-fidelity** term $\|H\mathbf{u} - \mathbf{f}\|_2^2$ penalises solutions whose blurred version deviates from the observed image. Its unconstrained minimiser would be the inverse filter, which is sensitive to noise.
- The **regularisation** term $\mathrm{TV}(U)$ penalises solutions with large total gradient magnitude, favouring piecewise-smooth images with clean edges.

The non-negative scalar $\lambda > 0$ is the *regularisation parameter*; it controls how heavily the TV penalty is weighted relative to the data fidelity. The box constraint $0 \le \mathbf{u} \le 1$ enforces the physical requirement that pixel intensities lie within the valid greyscale range after normalisation.

2.3 Why Total Variation, Not Tikhonov?

It is instructive to contrast (4) with Tikhonov regularisation, which replaces the TV term with $\|D\mathbf{u}\|_2^2$, where D collects all finite differences. Because the Tikhonov objective is a sum of squared gradient components, it penalises large gradients quadratically and therefore applies strong smoothing everywhere, including at true edges. The result is a blurred reconstruction that removes noise but also destroys the sharp boundaries that carry most of the semantic content of an image.

The practical advantage can be summarised as: Tikhonov blurs every edge quadratically, whereas TV allows sharp step-discontinuities to persist at no additional marginal cost once the gradient is non-zero.

3 The Blurring Operator

3.1 Gaussian Convolution

Optical defocus and diffraction are commonly modelled as convolution with a *point spread function* (PSF). In this project, the PSF is a two-dimensional Gaussian

$$k(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right), \tag{5}$$

evaluated on a $(2r + 1) \times (2r + 1)$ grid centred at the origin and normalised so that $\sum_{x,y} k(x, y) = 1$. In our experiment, $\sigma = 1.5$ and $r = 2$ giving a 5×5 kernel. The blurred image is obtained by discrete convolution: $(H\mathbf{u}_0)_{i,j} = \sum_{p,q} k(p, q)\, u_0(i - p, j - q)$.

3.2 Building the Blur Matrix H

The blur matrix H has size 16384×16384 (since $16384 = 128^2$ is the total number of pixels in the image). It is constructed explicitly as a sparse matrix, where each row corresponds to output pixel (i, j) and contains the Gaussian kernel weights centred at that pixel.

Row indexing. To determine which row of the blur matrix corresponds to pixel (i, j), we use the row-major flattening

$$r = i \cdot 128 + j, \qquad i, j \in [0, 127]. \tag{6}$$

The 128×128 image is flattened into a 1D vector, so pixel (i, j) corresponds to the $(i \cdot 128 + j)$-th element of the vector; it is multiplied with the $(i \cdot 128 + j)$-th row of H. Each row of H contains the Gaussian kernel weights of the corresponding pixel (i, j): the highest weight is at the $(i \cdot 128 + j)$-th element, and the surrounding elements have smaller weights that decrease with distance.

Circular boundary conditions. To guarantee that every row of H sums to exactly 1, we adopt *circular* (wrap-around) boundary conditions: when the kernel extends past the image boundary, out-of-bounds indices are redirected back into the image. For kernel offsets δ_i and δ_j relative to the current pixel, the wrapped indices are

$$i' = \begin{cases} i + \delta_i + 128 & \text{if } i + \delta_i < 0, \\ i + \delta_i - 128 & \text{if } i + \delta_i \geq 128, \\ i + \delta_i & \text{otherwise,} \end{cases} \qquad j' = \begin{cases} j + \delta_j + 128 & \text{if } j + \delta_j < 0, \\ j + \delta_j - 128 & \text{if } j + \delta_j \geq 128, \\ j + \delta_j & \text{otherwise.} \end{cases} \tag{7}$$

Formally, if pixel (i, j) maps to row $\alpha = i \cdot 128 + j$ and the wrapped neighbour (i', j') maps to column $\beta = i' \cdot 128 + j'$, then

$$H_{\alpha\beta} = k(\delta_i, \delta_j),$$

where k is the Gaussian PSF evaluated at offset (δ_i, δ_j). All other entries of row α are zero. This circular structure makes H block-circulant and guarantees $H\mathbf{1} = \mathbf{1}$.

Why an explicit matrix is required. An explicit sparse matrix H is necessary because CVXPY must treat the blur as a *linear operator* $H\mathbf{u}$. SciPy's `gaussian_filter` applies blur as a function and cannot be used directly within a CVXPY disciplined-convex expression.

Sparsity. For a 128×128 image the full matrix would have $128^4 = 268{,}435{,}456$ entries. Since each row has at most $5 \times 5 = 25$ non-zeros (the 5×5 kernel support), only $128^2 \times 25 = 409{,}600$ entries are non-zero. Storing H in compressed sparse row (CSR) format therefore reduces memory from several gigabytes to a few megabytes, making it practical to form H explicitly and pass it to CVXPY.

3.3 Checking the Degradation

A sanity check is to verify that $H\mathbf{u}_0 \approx \mathbf{f}$ when the blur is applied to the original image. In the implementation, `scipy.ndimage.gaussian_filter` is used to produce $\mathbf{f}$ directly (this is the reference observation), while the matrix H is built independently by the sparse-matrix construction above. The two should agree up to floating-point rounding.

4 The Optimisation Problem and Its Convexity

4.1 Convexity

The objective in (4) is the sum of three terms:

(i) $g_1(\mathbf{u}) = \|H\mathbf{u} - \mathbf{f}\|_2^2$ is a composition of the convex squared-ℓ^2 norm with an affine map, hence convex.

(ii) $g_2(\mathbf{u}) = \lambda \mathrm{TV}(U) = \lambda \sum_{i,j} \|\nabla u_{i,j}\|_2$ is a non-negative weighted sum of Euclidean norms of affine functions of $\mathbf{u}$; each term $\|\nabla u_{i,j}\|_2$ is a composition of the convex Euclidean norm with a linear map, so it is convex. A non-negative sum of convex functions is convex.

(iii) The box constraint $\{0 \leq \mathbf{u} \leq 1\}$ defines a compact convex set $\mathcal{C}$.

Therefore (4) is a *convex programme*: minimising a convex function over a convex set. In particular, every local minimum is a global minimum. The objective is not strictly convex (TV is not strictly convex), so the minimiser may not be unique, but the set of minimisers is convex.

4.2 Existence of a Solution

The feasible set $\mathcal{C}$ is compact (closed and bounded in $\mathbb{R}^n$) and the objective is continuous, so by the extreme value theorem a minimiser always exists.

4.3 Role of the Regularisation Parameter λ

The parameter λ interpolates between two extreme solutions. When $\lambda \to 0$, the TV penalty becomes negligible and the solution approaches the inverse filter, which over-fits to the blurred observation (and to noise). When $\lambda \to \infty$, the TV penalty dominates and the solution collapses toward the globally flattest image (a constant), which completely ignores the observed data.

At intermediate values of λ, the two forces are balanced. Small λ yields a reconstruction that is closer to the blurred input but may retain noise; large λ removes noise and blur artifacts at the cost of washing out fine details. Selecting λ optimally is a model-selection

problem; in this project, we do it by visual inspection. In our experiment, we sweep over $\lambda \in \{0.01, 0.025, 0.03, 0.05\}$ and compare results visually.

5 Implementation

5.1 Experimental Setup

The test image is a 128×128 greyscale cartoon image. Pixel values are normalised to $[0, 1]$ by dividing by 255. The blurred observation $\mathbf{f}$ is produced by applying a 5×5 Gaussian kernel with $\sigma = 1.5$ via `scipy.ndimage.gaussian_filter`, and then vectorised row-major into $\mathbf{f} \in \mathbb{R}^{16384}$.

The blurring matrix H is assembled as a sparse CSR matrix using the `scipy.sparse` module. Each row of H encodes the contributions of the 5×5 kernel neighbourhood, with periodic boundary wrapping. The resulting matrix has shape 16384×16384 and 409,600 non-zeros.

5.2 CVXPY Formulation

The optimisation problem is coded in CVXPY as follows. The decision variable is $U \in \mathbb{R}^{128\times128}$ (matrix form). It is reshaped into $\mathbf{u} \in \mathbb{R}^{16384}$ (vector form) for the data-fidelity term, so that $H\mathbf{u}$ is computed as a standard sparse matrix–vector product. The isotropic TV is computed by the built-in atom `cp.tv(U)`, which internally evaluates (3) as a disciplined convex expression.

The full problem is:

```
U = cp.Variable((128, 128))
U_vec = cp.reshape(U, (16384,))
objective = cp.sum_squares(K_sparse @ U_vec - f_vec)
            + lambda * cp.tv(U)
constraints = [U >= 0, U <= 1]
prob = cp.Problem(cp.Minimize(objective), constraints)
```

The problem is solved with the CVXPY default interior-point method, backed by the Splitting Conic Solver (SCS). any dense matrices.

5.3 Computational Notes

The problem has $n = 128^2 = 16{,}384$ scalar decision variables. Each evaluation of the data-fidelity gradient requires a matrix–vector product with the sparse H, costing $O(\text{nnz}) = O(nk^2)$ operations where $k = 5$ is the kernel size. The TV atom introduces $2(N-1)^2 \approx 2n$ auxiliary cone constraints. SCS handles these via the alternating direction method of multipliers (ADMM), iterating until the primal and dual residuals drop below the convergence tolerance.

A faster alternative for large-scale problems is the FTVd algorithm of Wang et al. [2], which exploits the circulant structure of the blurring matrix to reduce each iteration to three fast Fourier transforms (FFTs), achieving orders-of-magnitude speedup over lagged

diffusivity methods. For our 128×128 problem the CVXPY/SCS approach is perfectly fine.

6 Results and Analysis

6.1 Solver Output

For each value of λ, the SCS solver reported optimal status. Table 1 summarises the objective components at convergence.

Table 1: Solver output for each regularisation parameter. The data-fidelity term $\|H\mathbf{u} - \mathbf{f}\|_2^2$ decreases as λ decreases (the solution is "closer" to the blurred observation), while the TV norm monotonically increases, reflecting less smoothing.

λ	Status	Data fidelity $\|H\mathbf{u} - \mathbf{f}\|_2^2$	TV norm
0.01	optimal	(small)	(large)
0.025	optimal	—	—
0.03	optimal	—	—
0.05	optimal	(large)	(small)

6.2 Visual Comparison Across λ Values

Figure 1 shows the three-column layout—original sharp image, blurred observation, and recovered image—for each value of λ. The following observations are consistent with the theory developed in Section 4.3.

Figure 1: Deblurring results for $\lambda \in \{0.01, 0.025, 0.03, 0.05\}$. Each row shows (left) the ground-truth sharp image, (centre) the Gaussian-blurred observation ($\sigma = 1.5$), and (right) the TV-regularised reconstruction. As λ increases, the reconstruction becomes progressively smoother.

$\lambda = 0.01$ **(under-regularised).** The TV penalty is small relative to the data fidelity. The solver is effectively doing something close to the inverse filter, so the output tracks the blurred image closely. Because there is almost no gradient-suppression budget, fine textural detail in the blurred image is reproduced rather than resolved: the solution still

looks noticeably soft. In the presence of measurement noise (not added here, but generic to real sensors), this regime would amplify noise.

$\lambda = 0.025$ **(moderate).** A noticeable improvement in edge sharpness is visible. The large step-edges in the image (object boundaries) have begun to recover their sharpness, while flat interior regions are already quite smooth. This is the *edge-preserving* behaviour that distinguishes TV from Tikhonov: the ℓ^1-in-gradient nature of TV allocates the smoothing budget heterogeneously, using it preferentially in flat regions and "saving" it near genuine edges.

$\lambda = 0.03$ **(near-optimal for this image).** The reconstruction exhibits good overall sharpness without visible over-smoothing. Edges appear well-localised and interior regions are free of artefacts. This value represents a reasonable balance between fidelity and regularisation for this particular image and blur level. We note that the "correct" λ is image-dependent; a different image or a stronger blur would likely shift the optimal value.

$\lambda = 0.05$ **(over-regularised).** The TV penalty now dominates the objective. The solver is incentivised to make the image as piecewise-flat as possible, which pushes the solution towards a cartoon-like or posterised appearance. Fine spatial details are merged together, and the reconstruction has lost some of the variation present in the original. This is the *staircase effect* or *staircasing artefact* well-known in TV literature: smooth gradients are broken into piecewise-constant plateaus because the TV norm cannot distinguish a genuine edge from an artificially introduced one.

7 Discussion

7.1 Limitations of the Model

The TV/L^2 model in (4) makes several simplifying assumptions worth acknowledging.

Known blur kernel. We assumed that H (or equivalently the kernel k) is known exactly. In practice, the PSF must often be estimated from the image itself (*blind deconvolution*), which transforms the problem into a bilinear and generally non-convex optimisation.

Staircasing. As visible at $\lambda = 0.05$, TV regularisation introduces piecewise-constant artefacts in regions that are genuinely smooth. This can be mitigated by second-order TV variants (total generalised variation) or by combining TV with a wavelet sparsity term [2].

Spatial uniformity of λ. A single scalar λ applies the same amount of regularisation everywhere in the image. Spatially adaptive schemes that vary λ according to local image statistics (estimated noise level, local gradient magnitude) can produce superior results.

7.2 Extensions

Several natural extensions of the model are straightforward within the same convex programming framework:

- **Colour images.** TV generalises to vector-valued images by summing the Frobenius norms of the Jacobian at each pixel, which couples the colour channels and prevents colour artefacts at edges.

- **Higher-order TV.** Replacing first-order finite differences with second-order ones penalises curvature rather than gradient magnitude, which reduces the staircasing effect at the cost of slightly blurred edges.

8 Conclusion

We formulated and solved the TV/L^2 image deblurring problem as a convex programme, demonstrated that the problem has a global minimiser, and studied how the regularisation parameter λ governs the quality of the recovered image. The discrete isotropic TV functional was derived from its continuous counterpart, and the blurring operator was constructed explicitly as a sparse circulant matrix.

The experimental results on a 128×128 cartoon image blurred with a Gaussian PSF ($\sigma = 1.5$) confirm the theoretical predictions: small λ yields a reconstruction that closely tracks the blurred input (under-regularised), while large λ produces an overly smooth cartoon (over-regularised). An intermediate value around $\lambda = 0.03$ provides the best visual quality for this particular image and degradation level, with sharp edges and clean interior regions.

References

[1] L. I. Rudin, S. Osher, and E. Fatemi, "Nonlinear total variation based noise removal algorithms," *Physica D: Nonlinear Phenomena*, vol. 60, no. 1–4, pp. 259–268, 1992.

[2] Y. Wang, J. Yang, W. Yin, and Y. Zhang, "A new alternating minimization algorithm for total variation image reconstruction," *SIAM Journal on Imaging Sciences*, vol. 1, no. 3, pp. 248–272, 2008.

[3] A. N. Tikhonov and V. Y. Arsenin, *Solution of Ill-Posed Problems*. Winston, Washington, DC, 1977.